Relatório de Migração de Arquitetura de Dados

Migração do Projeto Analítico para Databricks Serverless via dbt

1. Visão Geral da Arquitetura

Este documento detalha o processo de migração e orquestração do projeto **portfolio_analytics**. A arquitetura foi desenhada para promover alta escalabilidade e governança de dados, estabelecendo o Databricks como a única fonte de verdade da organização.

Stack Tecnológico Adotado

- **Plataforma de Dados:** Databricks (Serverless SQL Warehouse)
- **Catálogo de Dados:** Unity Catalog
- **Formato de Armazenamento:** Delta Lake
- **Orquestração e Transformação:** dbt Core (Data Build Tool)

2. Implementação da Camada Bronze (Ingestão)

A primeira fase da migração consistiu na ingestão dos dados transacionais do ecossistema AdventureWorks para o ambiente cloud. O processo evitou o trânsito redundante de dados locais, extraindo o valor diretamente das fontes armazenadas em Volumes no Unity Catalog.

- Criação de um volume dedicado: `adventure_works_bronze`.
- Processamento dinâmico via PySpark para leitura dos arquivos brutos.
- **Higienização de Esquemas:** Implementação de uma rotina automatizada de Regex para remover caracteres inválidos (espaços, chaves, parênteses) dos nomes das colunas, garantindo compatibilidade com o formato Delta Lake e evitando erros de *parsing* SQL.
- Conversão bem-sucedida de todas as tabelas transacionais para tabelas governadas Delta, executadas em modo `overwrite` para assegurar a idempotência do pipeline.

3. Integração com dbt (Data Build Tool)

Com os dados alocados na plataforma, o ambiente de desenvolvimento local foi conectado ao warehouse remoto, habilitando a execução de transformações orientadas a modelos.

> *A conexão foi estabelecida via* `dbt-databricks`*, apontando o* `profiles.yml` *para o host e HTTP Path do cluster Serverless. A autenticação ocorreu com sucesso via token de acesso (DAPI), comprovada pela execução do comando* `dbt debug`.

3.1 Modelagem e Refatoração de Fontes

O arquivo `sources.yml` foi refatorado para apontar para as novas tabelas Delta no catálogo `workspace`, schema `default`. Para evitar a quebra dos modelos analíticos, utilizou-se o parâmetro `identifier` para mapear os nomes brutos do Databricks (ex: `sales_customer`) para os nomes lógicos esperados pelos modelos Staging (ex: `raw_sales.Customer`).

4. Lógica de Negócio e Governança Analítica (Camadas Silver e Gold)

A esteira de modelagem seguiu as melhores práticas analíticas, segregando responsabilidades entre as camadas Staging e Marts.

4.1 Padronização na Staging (Silver)

Os modelos da camada Staging foram materializados como *views* e assumiram a responsabilidade primária de padronização semântica. Identificadores técnicos e métricas em inglês foram traduzidos e expostos com nomenclatura de negócio, como a conversão de `LineTotal` para `receita_linha` e de `OrderQty` para `quantidade`.

4.2 Isolamento de Regras na Tabela Fato (Gold)

O modelo final `fato_rentabilidade_risco` foi materializado fisicamente (*table*) e concentrou o core das regras de negócio:

- **Filtro de Atividade:** Os cálculos de rentabilidade foram estritamente apontados para a tabela de serviços **encerrados**, garantindo que o risco seja mensurado sobre o histórico consolidado.

- **Isolamento de Variáveis:** As colunas de análise B2B (`FAIXA_INAD` e `COD_GRUPO`) foram mantidas como entidades estruturalmente diferentes e separadas, preservando a integridade das regras financeiras sem mesclagem indevida.

Resultado da Execução: 9 de 9 modelos compilados e materializados com sucesso (Status: SUCCESS).

5. Conclusão

A cadeia de transformação completa foi executada e validada, demonstrando fluxo contínuo desde o dado bruto no Delta Lake até o modelo final de consumo, consolidando a infraestrutura de Data Analytics do projeto na plataforma Databricks.